

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5

«Процедуры, функции, триггеры в PostgreSQL»

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Барецкий Максим Степанович

Факультет прикладной информатики

Группа К3241

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии

Преподаватель Говорова Марина Михайловна

Санкт-Петербург
2025/2026

1. Цель работы

Овладеть практическими навыками создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

2. Практическое задание

Для индивидуальной базы данных «Автомастерская» необходимо создать хранимые процедуры, функции и триггеры. Работа выполняется в SQL Shell psql.

1. Создать три хранимые процедуры для индивидуальной БД согласно варианту.
2. Создать необходимые триггеры для индивидуальной БД.
3. Проверить работу процедур и триггеров с помощью SQL-запросов.
4. Сделать подтверждающие скриншоты выполнения в psql.

Рисунок: Подключение к базе automasterskaya, установка search_path

```
[automasterskaya=# \c
psql (18.3 (Homebrew), server 15.17 (Homebrew))
You are now connected to database "automasterskaya" as user "bareckijmaksim".
automasterskaya=# █
```

3. Выполнение работы

3.1. Процедура повышения цены деталей Ford на 10%

Процедура увеличивает цену деталей, связанных с автомобилями Ford, на 10%. Детали выбираются по названию, каталожному номеру или производителю.

```
CREATE OR REPLACE PROCEDURE pr_increase_ford_parts_price()
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE part
    SET price = ROUND(price * 1.10, 2)
    WHERE part_name ILIKE '%Ford%'
       OR catalog_number ILIKE '%Ford%'
       OR manufacturer ILIKE '%Ford%';
END;
$$;
```

Рисунок: Создание процедуры pr_increase_ford_parts_price, сообщение CREATE PROCEDURE

```
automasterskaya=# CREATE OR REPLACE PROCEDURE pr_increase_ford_parts_price()
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE part
    SET price = ROUND(price * 1.10, 2)
    WHERE part_name ILIKE '%Ford%'
       OR catalog_number ILIKE '%Ford%'
       OR manufacturer ILIKE '%Ford%';
END;
[$$;
CREATE PROCEDURE
```

Проверка до выполнения процедуры:

```
SELECT part_id, part_name, catalog_number, manufacturer, price
FROM part
WHERE part_name ILIKE '%Ford%'
   OR catalog_number ILIKE '%Ford%'
   OR manufacturer ILIKE '%Ford%'
ORDER BY part_id;
```

Рисунок: список деталей Ford и их цена до вызова процедуры

```
automasterskaya=# SELECT part_id, part_name, catalog_number, manufacturer, price
FROM part
WHERE part_name ILIKE '%Ford%'
   OR catalog_number ILIKE '%Ford%'
   OR manufacturer ILIKE '%Ford%'
ORDER BY part_id;
```

part_id	part_name	catalog_number	manufacturer	price
3	Стойка стабилизатора	ST-2002	Lemforder	2400.00
4	Масляный фильтр Ford Focus	FORD-FLT-001	Ford	900.00
5	Свеча зажигания Ford Focus	FORD-SPK-002	Ford	650.00
6	Тормозные колодки Ford Mondeo	FORD-BRK-003	Ford	3500.00
10	Фильтр салона Ford Kuga	FORD-KUGA-CABIN-001	Ford	1200.00
11	Комплект свечей Ford Explorer	FORD-EXPL-SPARK-001	Ford	2800.00
15	Масляный фильтр Ford Focus	FORD-FOCUS-OIL-001	Ford	950.00
16	Воздушный фильтр Ford Kuga	FORD-KUGA-AIR-002	Ford	1350.00
17	Тормозные колодки Ford Mondeo	FORD-MON-BRK-003	Ford	4200.00
18	Свеча зажигания Ford Focus	FORD-FOCUS-SPK-004	Ford	750.00

(10 rows)

Вызов процедуры:

```
CALL pr_increase_ford_parts_price();
```

Рисунок: вызов процедуры

```
automasterskaya=# CALL pr_increase_ford_parts_price();
CALL
```

Проверка после выполнения процедуры:

```
SELECT part_id, part_name, catalog_number, manufacturer, price
FROM part
WHERE part_name ILIKE '%Ford%'
      OR catalog_number ILIKE '%Ford%'
      OR manufacturer ILIKE '%Ford%'
ORDER BY part_id;
```

Рисунок: цена деталей Ford увеличилась на 10%

```
automasterskaya=# SELECT part_id, part_name, catalog_number, manufacturer, price
FROM part
WHERE part_name ILIKE '%Ford%'
      OR catalog_number ILIKE '%Ford%'
      OR manufacturer ILIKE '%Ford%'
[ORDER BY part_id;
```

part_id	part_name	catalog_number	manufacturer	price
3	Стойка стабилизатора	ST-2002	Lemforder	2640.00
4	Масляный фильтр Ford Focus	FORD-FLT-001	Ford	990.00
5	Свеча зажигания Ford Focus	FORD-SPK-002	Ford	715.00
6	Тормозные колодки Ford Mondeo	FORD-BRK-003	Ford	3850.00
10	Фильтр салона Ford Kuga	FORD-KUGA-CABIN-001	Ford	1320.00
11	Комплект свечей Ford Explorer	FORD-EXPL-SPARK-001	Ford	3080.00
15	Масляный фильтр Ford Focus	FORD-FOCUS-OIL-001	Ford	1045.00
16	Воздушный фильтр Ford Kuga	FORD-KUGA-AIR-002	Ford	1485.00
17	Тормозные колодки Ford Mondeo	FORD-MON-BRK-003	Ford	4620.00
18	Свеча зажигания Ford Focus	FORD-FOCUS-SPK-004	Ford	825.00

(10 rows)

3.2. Процедура повышения разряда мастеров

Процедура повышает разряд тех мастеров, которые отремонтировали больше трех разных автомобилей. Подсчет выполняется по завершенным договорам.

```
CREATE OR REPLACE PROCEDURE pr_upgrade_master_grade()
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE employee_category ec
    SET grade = ((ec.grade)::int + 1)::varchar
    WHERE ec.grade ~ '^[0-9]+$'
    AND (ec.grade)::int < 6
    AND (
        ec.position_name ILIKE '%мастер%'
        OR ec.position_name ILIKE '%механик%'
    )
    AND ec.category_id IN (
        SELECT ea.category_id
        FROM employee_assignment ea
        JOIN contract_line cl ON cl.employee_id = ea.employee_id
        JOIN contract c ON c.contract_id = cl.contract_id
        WHERE c.actual_finish_date IS NOT NULL
        AND (ea.dismissal_date IS NULL OR ea.dismissal_date >= CURRENT_DATE)
        GROUP BY ea.category_id, ea.employee_id
        HAVING COUNT(DISTINCT c.car_id) > 3
    );
```

```
END;  
$$;
```

Рисунок: Создание процедуры pr_upgrade_master_grade, сообщение CREATE PROCEDURE

```
automasterskaya=# CREATE OR REPLACE PROCEDURE pr_upgrade_master_grade()  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    UPDATE employee_category ec  
    SET grade = ((ec.grade)::int + 1)::varchar  
    WHERE ec.grade ~ '^[0-9]+$'  
    AND (ec.grade)::int < 6  
    AND (  
        ec.position_name ILIKE '%мастер%'  
        OR ec.position_name ILIKE '%механик%'  
    )  
    AND ec.category_id IN (  
        SELECT ea.category_id  
        FROM employee_assignment ea  
        JOIN contract_line cl ON cl.employee_id = ea.employee_id  
        JOIN contract c ON c.contract_id = cl.contract_id  
        WHERE c.actual_finish_date IS NOT NULL  
        AND (ea.dismissal_date IS NULL OR ea.dismissal_date >= CURRENT_DATE)  
        GROUP BY ea.category_id, ea.employee_id  
        HAVING COUNT(DISTINCT c.car_id) > 3  
    );  
END;  
$$;  
CREATE PROCEDURE
```

Проверка до выполнения процедуры:

```
SELECT  
    e.employee_id,  
    e.full_name,  
    ec.category_id,  
    ec.position_name,  
    ec.specialization,  
    ec.grade,  
    COUNT(DISTINCT c.car_id) AS repaired_cars_count  
FROM employee e  
JOIN employee_assignment ea ON ea.employee_id = e.employee_id  
JOIN employee_category ec ON ec.category_id = ea.category_id  
LEFT JOIN contract_line cl ON cl.employee_id = e.employee_id  
LEFT JOIN contract c ON c.contract_id = cl.contract_id  
WHERE ec.position_name ILIKE '%механик%'  
    OR ec.position_name ILIKE '%мастер%'  
GROUP BY e.employee_id, e.full_name, ec.category_id, ec.position_name, ec.specialization, ec.grade  
ORDER BY repaired_cars_count DESC;
```

Рисунок: количество отремонтированных автомобилей и разряды мастеров до процедуры

```
automasterskaya=# SELECT
    e.employee_id,
    e.full_name,
    ec.category_id,
    ec.position_name,
    ec.specialization,
    ec.grade,
    COUNT(DISTINCT c.car_id) AS repaired_cars_count
FROM employee e
JOIN employee_assignment ea ON ea.employee_id = e.employee_id
JOIN employee_category ec ON ec.category_id = ea.category_id
LEFT JOIN contract_line cl ON cl.employee_id = e.employee_id
LEFT JOIN contract c ON c.contract_id = cl.contract_id
WHERE ec.position_name ILIKE '%механик%'
    OR ec.position_name ILIKE '%мастер%'
GROUP BY e.employee_id, e.full_name, ec.category_id, ec.position_name, ec.specialization, ec.grade
ORDER BY repaired_cars_count DESC;
```

employee_id	full_name	category_id	position_name	specialization	grade	repaired_cars_count
2	Петров Алексей Николаевич	2	Мастер	Двигатель	5	5
3	Сидоров Максим Олегович	3	Мастер	Ходовая часть	4	1
4	Кузнецов Илья Павлович	4	Мастер	Диагностика	5	1

(3 rows)

```
CALL pr_upgrade_master_grade();
```

Рисунок: Вызов процедуры

```
[automasterskaya=# CALL pr_upgrade_master_grade();
CALL
```

Проверка после выполнения процедуры:

```
SELECT
    e.employee_id,
    e.full_name,
    ec.category_id,
    ec.position_name,
    ec.specialization,
    ec.grade,
    COUNT(DISTINCT c.car_id) AS repaired_cars_count
FROM employee e
JOIN employee_assignment ea ON ea.employee_id = e.employee_id
JOIN employee_category ec ON ec.category_id = ea.category_id
LEFT JOIN contract_line cl ON cl.employee_id = e.employee_id
LEFT JOIN contract c ON c.contract_id = cl.contract_id
WHERE ec.position_name ILIKE '%механик%'
    OR ec.position_name ILIKE '%мастер%'
GROUP BY e.employee_id, e.full_name, ec.category_id, ec.position_name, ec.specialization, ec.grade
ORDER BY repaired_cars_count DESC;
```


Рисунок: разряд мастера увеличился

```
automasterskaya=# CALL pr_upgrade_master_grade();
CALL
automasterskaya=# SELECT
    e.employee_id,
    e.full_name,
    ec.category_id,
    ec.position_name,
    ec.specialization,
    ec.grade,
    COUNT(DISTINCT c.car_id) AS repaired_cars_count
FROM employee e
JOIN employee_assignment ea ON ea.employee_id = e.employee_id
JOIN employee_category ec ON ec.category_id = ea.category_id
LEFT JOIN contract_line cl ON cl.employee_id = e.employee_id
LEFT JOIN contract c ON c.contract_id = cl.contract_id
WHERE ec.position_name ILIKE '%механик%'
    OR ec.position_name ILIKE '%мастер%'
GROUP BY e.employee_id, e.full_name, ec.category_id, ec.position_name, ec.specialization, ec.grade
ORDER BY repaired_cars_count DESC;
```

employee_id	full_name	category_id	position_name	specialization	grade	repaired_cars_count
2	Петров Алексей Николаевич	2	Мастер	Двигатель	6	5
3	Сидоров Максим Олегович	3	Мастер	Ходовая часть	4	1
4	Кузнецов Илья Павлович	4	Мастер	Диагностика	5	1

(3 rows)

3.3. Процедура подсчета автомобилей за истекший квартал

Процедура формирует временную таблицу с количеством автомобилей, отремонтированных каждым механиком за истекший квартал.

```
CREATE OR REPLACE PROCEDURE pr_repaired_cars_by_mechanics_last_quarter()
LANGUAGE plpgsql
AS $$
BEGIN
    DROP TABLE IF EXISTS tmp_mechanic_quarter_result;

    CREATE TEMP TABLE tmp_mechanic_quarter_result AS
    SELECT
        e.employee_id,
        e.full_name AS mechanic_name,
        ec.position_name,
        ec.specialization,
        ec.grade,
        COUNT(DISTINCT c.car_id) AS repaired_cars_count
    FROM employee e
    JOIN employee_assignment ea ON ea.employee_id = e.employee_id
    JOIN employee_category ec ON ec.category_id = ea.category_id
    LEFT JOIN contract_line cl ON cl.employee_id = e.employee_id
    LEFT JOIN contract c
        ON c.contract_id = cl.contract_id
        AND c.actual_finish_date >= date_trunc('quarter', CURRENT_DATE) - INTERVAL '3 months'
        AND c.actual_finish_date < date_trunc('quarter', CURRENT_DATE)
    WHERE (
        ec.position_name ILIKE '%мастер%'
        OR ec.position_name ILIKE '%механик%'
    )
    AND (ea.dismissal_date IS NULL OR ea.dismissal_date >= CURRENT_DATE)
    GROUP BY
        e.employee_id,
        e.full_name,
        ec.position_name,
```

```

        ec.specialization,
        ec.grade
    ORDER BY repaired_cars_count DESC, mechanic_name;
END;
$$;

automasterskaya=# CREATE OR REPLACE PROCEDURE pr_repaired_cars_by_mechanics_last_quarter()
LANGUAGE plpgsql
AS $$
BEGIN
    DROP TABLE IF EXISTS tmp_mechanic_quarter_result;

    CREATE TEMP TABLE tmp_mechanic_quarter_result AS
    SELECT
        e.employee_id,
        e.full_name AS mechanic_name,
        ec.position_name,
        ec.specialization,
        ec.grade,
        COUNT(DISTINCT c.car_id) AS repaired_cars_count
    FROM employee e
    JOIN employee_assignment ea ON ea.employee_id = e.employee_id
    JOIN employee_category ec ON ec.category_id = ea.category_id
    LEFT JOIN contract_line cl ON cl.employee_id = e.employee_id
    LEFT JOIN contract c
        ON c.contract_id = cl.contract_id
        AND c.actual_finish_date >= date_trunc('quarter', CURRENT_DATE) - INTERVAL '3 months'
        AND c.actual_finish_date < date_trunc('quarter', CURRENT_DATE)
    WHERE (
        ec.position_name ILIKE '%мастер%'
        OR ec.position_name ILIKE '%механик%'
    )
    AND (ea.dismissal_date IS NULL OR ea.dismissal_date >= CURRENT_DATE)
    GROUP BY
        e.employee_id,
        e.full_name,
        ec.position_name,
        ec.specialization,
        ec.grade
    ORDER BY repaired_cars_count DESC, mechanic_name;
END;
$$;
CREATE PROCEDURE

CALL pr_repaired_cars_by_mechanics_last_quarter();

SELECT *
FROM tmp_mechanic_quarter_result;

```

Рисунок: временная таблица tmp_mechanic_quarter_result

```

automasterskaya=# SELECT *
FROM tmp_mechanic_quarter_result;

```

employee_id	mechanic_name	position_name	specialization	grade	repaired_cars_count
4	Кузнецов Илья Павлович	Мастер	Диагностика	5	0
2	Петров Алексей Николаевич	Мастер	Двигатель	6	0
3	Сидоров Максим Олегович	Мастер	Ходовая часть	4	0

(3 rows)

4. Триггеры

4.1. Триггер проверки дат договора

```
CREATE OR REPLACE FUNCTION fn_check_contract_dates()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.acceptance_date < NEW.request_date THEN
        RAISE EXCEPTION 'Дата принятия автомобиля не может быть раньше даты заказа';
    END IF;

    IF NEW.planned_finish_date IS NOT NULL
        AND NEW.planned_finish_date < NEW.acceptance_date THEN
        RAISE EXCEPTION 'Плановая дата окончания ремонта не может быть раньше даты принятия автомобиля';
    END IF;

    IF NEW.actual_finish_date IS NOT NULL
        AND NEW.actual_finish_date < NEW.acceptance_date THEN
        RAISE EXCEPTION 'Фактическая дата окончания ремонта не может быть раньше даты принятия автомобиля';
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_check_contract_dates ON contract;

CREATE TRIGGER trg_check_contract_dates
BEFORE INSERT OR UPDATE ON contract
FOR EACH ROW
EXECUTE PROCEDURE fn_check_contract_dates();
```

Рисунок: Создание функции fn_check_contract_dates и триггера trg_check_contract_dates

```
automasterskaya=# CREATE OR REPLACE FUNCTION fn_check_contract_dates()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.acceptance_date < NEW.request_date THEN
        RAISE EXCEPTION 'Дата принятия автомобиля не может быть раньше даты заказа';
    END IF;

    IF NEW.planned_finish_date IS NOT NULL
    AND NEW.planned_finish_date < NEW.acceptance_date THEN
        RAISE EXCEPTION 'Плановая дата окончания ремонта не может быть раньше даты принятия автомобиля';
    END IF;

    IF NEW.actual_finish_date IS NOT NULL
    AND NEW.actual_finish_date < NEW.acceptance_date THEN
        RAISE EXCEPTION 'Фактическая дата окончания ремонта не может быть раньше даты принятия автомобиля';
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_check_contract_dates ON contract;

CREATE TRIGGER trg_check_contract_dates
BEFORE INSERT OR UPDATE ON contract
FOR EACH ROW
EXECUTE PROCEDURE fn_check_contract_dates();
CREATE FUNCTION
NOTICE: trigger "trg_check_contract_dates" for relation "contract" does not exist, skipping
DROP TRIGGER
CREATE TRIGGER
automasterskaya=#
```

```
INSERT INTO contract (
    workshop_id,
    client_id,
    car_id,
    administrator_employee_id,
    request_date,
    acceptance_date,
    planned_finish_date,
    actual_finish_date,
    contract_status,
    payment_status
)
VALUES (
    (SELECT workshop_id FROM workshop ORDER BY workshop_id LIMIT 1),
    (SELECT client_id FROM client ORDER BY client_id LIMIT 1),
    (SELECT car_id FROM car ORDER BY car_id LIMIT 1),
    (SELECT employee_id FROM employee ORDER BY employee_id LIMIT 1),
    DATE '2024-05-10',
    DATE '2024-05-09',
    DATE '2024-05-12',
    NULL,
    'новый',
    'не оплачено'
);
```

Рисунок: Проверка триггера дат: ошибка при попытке указать acceptance_date раньше request_date

```

CREATE TRIGGER
automasterskaya=# INSERT INTO contract (
    workshop_id,
    client_id,
    car_id,
    administrator_employee_id,
    request_date,
    acceptance_date,
    planned_finish_date,
    actual_finish_date,
    contract_status,
    payment_status
)
VALUES (
    (SELECT workshop_id FROM workshop ORDER BY workshop_id LIMIT 1),
    (SELECT client_id FROM client ORDER BY client_id LIMIT 1),
    (SELECT car_id FROM car ORDER BY car_id LIMIT 1),
    (SELECT employee_id FROM employee ORDER BY employee_id LIMIT 1),
    DATE '2024-05-10',
    DATE '2024-05-09',
    DATE '2024-05-12',
    NULL,
    'новый',
    'не оплачено'
);
ERROR: Дата принятия автомобиля не может быть раньше даты заказа
CONTEXT: PL/pgSQL function fn_check_contract_dates() line 4 at RAISE

```

4.2. Триггер проверки администратора договора

```

CREATE OR REPLACE FUNCTION fn_check_contract_administrator()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_count int;
BEGIN
    SELECT COUNT(*)
    INTO v_count
    FROM employee_assignment ea
    JOIN employee_category ec ON ec.category_id = ea.category_id
    WHERE ea.employee_id = NEW.administrator_employee_id
    AND ec.position_name ILIKE '%администратор%'
    AND ea.appointment_date <= NEW.request_date
    AND (ea.dismissal_date IS NULL OR ea.dismissal_date >= NEW.request_date);

    IF v_count = 0 THEN
        RAISE EXCEPTION 'Договор может сопровождать только действующий администратор';
    END IF;

    RETURN NEW;

```

```
END;  
$$;
```

```
DROP TRIGGER IF EXISTS trg_check_contract_administrator ON contract;
```

```
CREATE TRIGGER trg_check_contract_administrator  
BEFORE INSERT OR UPDATE ON contract  
FOR EACH ROW  
EXECUTE PROCEDURE fn_check_contract_administrator();
```

Рисунок: Создание функции `fn_check_contract_administrator` и триггера `trg_check_contract_administrator`

```
automasterskaya=# CREATE OR REPLACE FUNCTION fn_check_contract_administrator()  
RETURNS TRIGGER  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_count int;  
BEGIN  
    SELECT COUNT(*)  
    INTO v_count  
    FROM employee_assignment ea  
    JOIN employee_category ec ON ec.category_id = ea.category_id  
    WHERE ea.employee_id = NEW.administrator_employee_id  
        AND ec.position_name ILIKE '%администратор%'  
        AND ea.appointment_date <= NEW.request_date  
        AND (ea.dismissal_date IS NULL OR ea.dismissal_date >= NEW.request_date);  
  
    IF v_count = 0 THEN  
        RAISE EXCEPTION 'Договор может сопровождать только действующий администратор';  
    END IF;  
  
    RETURN NEW;  
END;  
$$;  
  
DROP TRIGGER IF EXISTS trg_check_contract_administrator ON contract;  
  
CREATE TRIGGER trg_check_contract_administrator  
BEFORE INSERT OR UPDATE ON contract  
FOR EACH ROW  
EXECUTE PROCEDURE fn_check_contract_administrator();  
CREATE FUNCTION  
NOTICE: trigger "trg_check_contract_administrator" for relation "contract" does not exist, skipping  
DROP TRIGGER  
CREATE TRIGGER  
=
```

Рисунок: Проверка триггера администратора: попытка создать договор с мастером вместо администратора завершается ошибкой

```
automasterskaya=# INSERT INTO contract (
    workshop_id,
    client_id,
    car_id,
    administrator_employee_id,
    request_date,
    acceptance_date,
    planned_finish_date,
    actual_finish_date,
    contract_status,
    payment_status
)
VALUES (
    (SELECT workshop_id FROM workshop ORDER BY workshop_id LIMIT 1),
    (SELECT client_id FROM client ORDER BY client_id LIMIT 1),
    (SELECT car_id FROM car ORDER BY car_id LIMIT 1),
    (
        SELECT e.employee_id
        FROM employee e
        JOIN employee_assignment ea ON ea.employee_id = e.employee_id
        JOIN employee_category ec ON ec.category_id = ea.category_id
        WHERE ec.position_name ILIKE '%мастер%'
            OR ec.position_name ILIKE '%механик%'
        LIMIT 1
    ),
    CURRENT_DATE,
    CURRENT_DATE,
    CURRENT_DATE + 5,
    NULL,
    'новый',
    'не оплачено'
);
ERROR: Договор может сопровождать только действующий администратор
CONTEXT: PL/pgSQL function fn_check_contract_administrator() line 15 at RAISE
automasterskaya=#
```

4.3. Триггер проверки исполнителя работ

```
CREATE OR REPLACE FUNCTION fn_check_contract_line_mechanic()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_count int;
BEGIN
    SELECT COUNT(*)
    INTO v_count
    FROM employee_assignment ea
    JOIN employee_category ec ON ec.category_id = ea.category_id
    WHERE ea.employee_id = NEW.employee_id
    AND (
        ec.position_name ILIKE '%мастер%'
        OR ec.position_name ILIKE '%механик%'
    )
    AND ea.appointment_date <= CURRENT_DATE
```



```

AND (ea.dismissal_date IS NULL OR ea.dismissal_date >= CURRENT_DATE);

IF v_count = 0 THEN
    RAISE EXCEPTION 'Вид работ может выполнять только действующий мастер или механик';
END IF;

RETURN NEW;
END;
$$;

```

```

DROP TRIGGER IF EXISTS trg_check_contract_line_mechanic ON contract_line;

```

```

CREATE TRIGGER trg_check_contract_line_mechanic
BEFORE INSERT OR UPDATE ON contract_line
FOR EACH ROW
EXECUTE PROCEDURE fn_check_contract_line_mechanic();

```

Рисунок: Создание функции fn_check_contract_line_mechanic и триггера trg_check_contract_line_mechanic

```

automasterskaya=# CREATE OR REPLACE FUNCTION fn_check_contract_line_mechanic()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_count int;
BEGIN
    SELECT COUNT(*)
    INTO v_count
    FROM employee_assignment ea
    JOIN employee_category ec ON ec.category_id = ea.category_id
    WHERE ea.employee_id = NEW.employee_id
    AND (
        ec.position_name ILIKE '%мастер%'
        OR ec.position_name ILIKE '%механик%'
    )
    AND ea.appointment_date <= CURRENT_DATE
    AND (ea.dismissal_date IS NULL OR ea.dismissal_date >= CURRENT_DATE);

    IF v_count = 0 THEN
        RAISE EXCEPTION 'Вид работ может выполнять только действующий мастер или механик';
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_check_contract_line_mechanic ON contract_line;

CREATE TRIGGER trg_check_contract_line_mechanic
BEFORE INSERT OR UPDATE ON contract_line
FOR EACH ROW
EXECUTE PROCEDURE fn_check_contract_line_mechanic();
CREATE FUNCTION
NOTICE: trigger "trg_check_contract_line_mechanic" for relation "contract_line" does not exist, skipping
DROP TRIGGER
CREATE TRIGGER

```

Рисунок: Проверка триггера исполнителя: попытка назначить работу администратору завершается ошибкой

```
-----
automasterskaya=# INSERT INTO contract_line (
    contract_id,
    employee_id,
    service_type_id,
    labor_price
)
VALUES (
    (SELECT contract_id FROM contract ORDER BY contract_id LIMIT 1),
    (
        SELECT e.employee_id
        FROM employee e
        JOIN employee_assignment ea ON ea.employee_id = e.employee_id
        JOIN employee_category ec ON ec.category_id = ea.category_id
        WHERE ec.position_name = 'Администратор'
        LIMIT 1
    ),
    (SELECT service_type_id FROM service_type ORDER BY service_type_id LIMIT 1),
    1000
);
ERROR: Вид работ может выполнять только действующий мастер или механик
CONTEXT: PL/pgSQL function fn_check_contract_line_mechanic() line 18 at RAISE
```

4.4. Триггер автозаполнения стоимости работы

```
CREATE OR REPLACE FUNCTION fn_fill_labor_price()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_labor_price numeric(10,2);
BEGIN
    IF NEW.labor_price IS NULL OR NEW.labor_price = 0 THEN
        SELECT ROUND(sp.hour_price * sp.norm_hours, 2)
        INTO v_labor_price
        FROM service_price sp
        LEFT JOIN price_period pp ON pp.price_id = sp.price_id
        WHERE sp.service_type_id = NEW.service_type_id
        AND (
            pp.period_id IS NULL
            OR (
                pp.start_date <= CURRENT_DATE
                AND (pp.end_date IS NULL OR pp.end_date >= CURRENT_DATE)
            )
        )
        ORDER BY pp.start_date DESC NULLS LAST, sp.price_id DESC
        LIMIT 1;

        IF v_labor_price IS NULL THEN
            RAISE EXCEPTION 'Не найден актуальный прайс для выбранного вида работ';
        END IF;

        NEW.labor_price := v_labor_price;
    END IF;

    RETURN NEW;
END;
```

```
$$;
```

```
DROP TRIGGER IF EXISTS trg_fill_labor_price ON contract_line;
```

```
CREATE TRIGGER trg_fill_labor_price  
BEFORE INSERT OR UPDATE ON contract_line  
FOR EACH ROW  
EXECUTE PROCEDURE fn_fill_labor_price();
```

Рисунок: Создание функции fn_fill_labor_price и триггера trg_fill_labor_price

```
automasterskaya=# CREATE OR REPLACE FUNCTION fn_fill_labor_price()  
RETURNS TRIGGER  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_labor_price numeric(10,2);  
BEGIN  
    IF NEW.labor_price IS NULL OR NEW.labor_price = 0 THEN  
        SELECT ROUND(sp.hour_price * sp.norm_hours, 2)  
        INTO v_labor_price  
        FROM service_price sp  
        LEFT JOIN price_period pp ON pp.price_id = sp.price_id  
        WHERE sp.service_type_id = NEW.service_type_id  
        AND (  
            pp.period_id IS NULL  
            OR (  
                pp.start_date <= CURRENT_DATE  
                AND (pp.end_date IS NULL OR pp.end_date >= CURRENT_DATE)  
            )  
        )  
        ORDER BY pp.start_date DESC NULLS LAST, sp.price_id DESC  
        LIMIT 1;  
  
        IF v_labor_price IS NULL THEN  
            RAISE EXCEPTION 'Не найден актуальный прайс для выбранного вида работ';  
        END IF;  
  
        NEW.labor_price := v_labor_price;  
    END IF;  
  
    RETURN NEW;  
END;  
$$;  
  
DROP TRIGGER IF EXISTS trg_fill_labor_price ON contract_line;  
  
CREATE TRIGGER trg_fill_labor_price  
BEFORE INSERT OR UPDATE ON contract_line  
FOR EACH ROW  
EXECUTE PROCEDURE fn_fill_labor_price();  
CREATE FUNCTION  
NOTICE: trigger "trg_fill_labor_price" for relation "contract_line" does not exist, skipping  
DROP TRIGGER  
CREATE TRIGGER _
```

Рисунок: ДО: последние строки contract_line до вставки строки с labor_price = 0

```
automasterskaya=# SELECT line_id, contract_id, employee_id, service_type_id, labor_price
FROM contract_line
ORDER BY line_id DESC
LIMIT 5;
```

line_id	contract_id	employee_id	service_type_id	labor_price
11	8	2	1	3000.00
10	7	2	1	3000.00
9	6	2	1	3000.00
8	2	2	1	3000.00
5	2	4	3	1800.00

(5 rows)

Рисунок: ПОСЛЕ: новая строка contract_line, где labor_price автоматически заменен на цену из прайса

```
automasterskaya=# SELECT line_id, contract_id, employee_id, service_type_id, labor_price
FROM contract_line
ORDER BY line_id DESC
LIMIT 5;
```

line_id	contract_id	employee_id	service_type_id	labor_price
13	1	3	1	1800.00
11	8	2	1	3000.00
10	7	2	1	3000.00
9	6	2	1	3000.00
8	2	2	1	3000.00

(5 rows)

4.5. Триггер автозаполнения цены использованной детали

```
CREATE OR REPLACE FUNCTION fn_fill_used_part_price()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_part_price numeric(10,2);
BEGIN
    IF lower(trim(NEW.supply_source)) = 'клиент' THEN
        NEW.price_at_repair := 0;

    ELSIF lower(trim(NEW.supply_source)) = 'автомастерская' THEN
        SELECT price
        INTO v_part_price
        FROM part
        WHERE part_id = NEW.part_id;

        IF v_part_price IS NULL THEN
            RAISE EXCEPTION 'Деталь с указанным part_id не найдена';
        END IF;

        NEW.price_at_repair := v_part_price;

    ELSE
        RAISE EXCEPTION 'Источник детали должен быть клиент или автомастерская';
    END IF;
END IF;
```



```

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_fill_used_part_price ON used_part;

```

```

CREATE TRIGGER trg_fill_used_part_price
BEFORE INSERT OR UPDATE ON used_part
FOR EACH ROW
EXECUTE PROCEDURE fn_fill_used_part_price();

```

Рисунок: Создание функции fn_fill_used_part_price и триггера trg_fill_used_part_price

```

automasterskaya=# CREATE OR REPLACE FUNCTION fn_fill_used_part_price()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_part_price numeric(10,2);
BEGIN
    IF lower(trim(NEW.supply_source)) = 'клиент' THEN
        NEW.price_at_repair := 0;

    ELSIF lower(trim(NEW.supply_source)) = 'автомастерская' THEN
        SELECT price
        INTO v_part_price
        FROM part
        WHERE part_id = NEW.part_id;

        IF v_part_price IS NULL THEN
            RAISE EXCEPTION 'Деталь с указанным part_id не найдена';
        END IF;

        NEW.price_at_repair := v_part_price;

    ELSE
        RAISE EXCEPTION 'Источник детали должен быть клиент или автомастерская';
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_fill_used_part_price ON used_part;

CREATE TRIGGER trg_fill_used_part_price
BEFORE INSERT OR UPDATE ON used_part
FOR EACH ROW
EXECUTE PROCEDURE fn_fill_used_part_price();
CREATE FUNCTION
NOTICE: trigger "trg_fill_used_part_price" for relation "used_part" does not exist, skipping
DROP TRIGGER
CREATE TRIGGER
_

```


Рисунок: ДО: строки used_part до изменения источника детали

```
automasterskaya=# SELECT used_part_id, line_id, part_id, quantity, supply_source, price_at_repair
FROM used_part
ORDER BY used_part_id
LIMIT 5;
```

used_part_id	line_id	part_id	quantity	supply_source	price_at_repair
1	1	1	1	автомастерская	650.00
2	1	2	1	автомастерская	3200.00
4	1	7	1	автомастерская	1100.00
5	2	4	1	автомастерская	900.00
6	3	5	4	клиент	0.00

(5 rows)

Рисунок: ПОСЛЕ: при supply_source = клиент цена price_at_repair автоматически стала 0

```
automasterskaya=# SELECT used_part_id, line_id, part_id, quantity, supply_source, price_at_repair
FROM used_part
ORDER BY used_part_id
[LIMIT 5;
```

used_part_id	line_id	part_id	quantity	supply_source	price_at_repair
1	1	1	1	клиент	0.00
2	1	2	1	автомастерская	3200.00
4	1	7	1	автомастерская	1100.00
5	2	4	1	автомастерская	900.00
6	3	5	4	клиент	0.00

(5 rows)

4.6. Триггер аудита строк договора

```
CREATE TABLE IF NOT EXISTS contract_line_audit (
  audit_id integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  operation_type varchar(20) NOT NULL,
  line_id integer,
  contract_id integer,
  employee_id integer,
  service_type_id integer,
  old_labor_price numeric(10,2),
  new_labor_price numeric(10,2),
  changed_at timestamp NOT NULL DEFAULT now()
);
```

```
CREATE OR REPLACE FUNCTION fn_audit_contract_line()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
  IF TG_OP = 'INSERT' THEN
    INSERT INTO contract_line_audit (
      operation_type, line_id, contract_id, employee_id,
      service_type_id, old_labor_price, new_labor_price
    )
    VALUES (
      'INSERT', NEW.line_id, NEW.contract_id, NEW.employee_id,
      NEW.service_type_id, NULL, NEW.labor_price
    );
  RETURN NEW;
END;
```

```
ELSIF TG_OP = 'UPDATE' THEN
    INSERT INTO contract_line_audit (
        operation_type, line_id, contract_id, employee_id,
        service_type_id, old_labor_price, new_labor_price
    )
    VALUES (
        'UPDATE', NEW.line_id, NEW.contract_id, NEW.employee_id,
        NEW.service_type_id, OLD.labor_price, NEW.labor_price
    );
    RETURN NEW;
```

```
ELSIF TG_OP = 'DELETE' THEN
    INSERT INTO contract_line_audit (
        operation_type, line_id, contract_id, employee_id,
        service_type_id, old_labor_price, new_labor_price
    )
    VALUES (
        'DELETE', OLD.line_id, OLD.contract_id, OLD.employee_id,
        OLD.service_type_id, OLD.labor_price, NULL
    );
    RETURN OLD;
END IF;
```

```
RETURN NULL;
END;
$$;
```

```
DROP TRIGGER IF EXISTS trg_audit_contract_line ON contract_line;
```

```
CREATE TRIGGER trg_audit_contract_line
AFTER INSERT OR UPDATE OR DELETE ON contract_line
FOR EACH ROW
EXECUTE PROCEDURE fn_audit_contract_line();
```

Рисунок: Создание таблицы contract_line_audit, функции fn_audit_contract_line и триггера trg_audit_contract_line

```
CREATE OR REPLACE FUNCTION fn_audit_contract_line()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO contract_line_audit (
            operation_type,
            line_id,
            contract_id,
            employee_id,
            service_type_id,
            old_labor_price,
            new_labor_price
        )
        VALUES (
            'INSERT',
            NEW.line_id,
            NEW.contract_id,
            NEW.employee_id,
            NEW.service_type_id,
            NULL,
            NEW.labor_price
        );

        RETURN NEW;

    ELSIF TG_OP = 'UPDATE' THEN
        INSERT INTO contract_line_audit (
            operation_type,
            line_id,
            contract_id,
            employee_id,
            service_type_id,
            old_labor_price,
            new_labor_price
        )
        VALUES (
            'UPDATE',
            NEW.line_id,
            NEW.contract_id,
            NEW.employee_id,
            NEW.service_type_id,
            OLD.labor_price,
            NEW.labor_price
        );

        RETURN NEW;

    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO contract_line_audit (
            operation_type,
            line_id,
            contract_id,
            employee_id,
            service_type_id,
            old_labor_price,
            new_labor_price
        )
        VALUES (
            'DELETE',
            OLD.line_id,
            OLD.contract_id,
            OLD.employee_id,
            OLD.service_type_id,
            OLD.labor_price,
            NULL
        );

        RETURN OLD;
    END IF;

    RETURN NULL;
END;
$$;

DROP TRIGGER IF EXISTS trg_audit_contract_line ON contract_line;

CREATE TRIGGER trg_audit_contract_line
AFTER INSERT OR UPDATE OR DELETE ON contract_line
FOR EACH ROW
```

Рисунок: ДО: таблица contract_line_audit до изменения строки contract_line

```
automasterskaya=# SELECT *
FROM contract_line_audit
ORDER BY audit_id;
 audit_id | operation_type | line_id | contract_id | employee_id | service_type_id | old_labor_price | new_labor_price | changed_at
-----+-----+-----+-----+-----+-----+-----+-----+-----
(0 rows)
```

Рисунок: ПОСЛЕ: после UPDATE появилась запись аудита со старой и новой ценой

```
automasterskaya=# SELECT *
FROM contract_line_audit
ORDER BY audit_id;
 audit_id | operation_type | line_id | contract_id | employee_id | service_type_id | old_labor_price | new_labor_price | changed_at
-----+-----+-----+-----+-----+-----+-----+-----+-----
1 | UPDATE | 1 | 1 | 2 | 1 | 1800.00 | 2300.00 | 2026-05-24 15:43:43.26938
(1 row)
```

4.7. Триггер запрета пересекающихся назначений

```
CREATE OR REPLACE FUNCTION fn_check_employee_assignment_overlap()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_count int;
BEGIN
    SELECT COUNT(*)
    INTO v_count
    FROM employee_assignment ea
    WHERE ea.employee_id = NEW.employee_id
        AND ea.assignment_id <> COALESCE(NEW.assignment_id, -1)
        AND NEW.appointment_date <= COALESCE(ea.dismissal_date, DATE '9999-12-31')
        AND COALESCE(NEW.dismissal_date, DATE '9999-12-31') >= ea.appointment_date;

    IF v_count > 0 THEN
        RAISE EXCEPTION 'У сотрудника уже есть назначение на должность в указанный период!';
    END IF;

    RETURN NEW;
END;
$$;
```

```
DROP TRIGGER IF EXISTS trg_check_employee_assignment_overlap ON employee_assignment;
```

```
CREATE TRIGGER trg_check_employee_assignment_overlap
BEFORE INSERT OR UPDATE ON employee_assignment
FOR EACH ROW
EXECUTE PROCEDURE fn_check_employee_assignment_overlap();
```

Рисунок: Создание функции fn_check_employee_assignment_overlap и триггера trg_check_employee_assignment_overlap

```
automasterskaya=# CREATE OR REPLACE FUNCTION fn_check_employee_assignment_overlap()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_count int;
BEGIN
    SELECT COUNT(*)
    INTO v_count
    FROM employee_assignment ea
    WHERE ea.employee_id = NEW.employee_id
    AND ea.assignment_id <> COALESCE(NEW.assignment_id, -1)
    AND NEW.appointment_date <= COALESCE(ea.dismissal_date, DATE '9999-12-31')
    AND COALESCE(NEW.dismissal_date, DATE '9999-12-31') >= ea.appointment_date;

    IF v_count > 0 THEN
        RAISE EXCEPTION 'У сотрудника уже есть назначение на должность в указанный период';
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_check_employee_assignment_overlap ON employee_assignment;

CREATE TRIGGER trg_check_employee_assignment_overlap
BEFORE INSERT OR UPDATE ON employee_assignment
FOR EACH ROW
EXECUTE PROCEDURE fn_check_employee_assignment_overlap();
CREATE FUNCTION
NOTICE: trigger "trg_check_employee_assignment_overlap" for relation "employee_assignment" does not exist, skipping
DROP TRIGGER
CREATE TRIGGER
.. ■
```

Рисунок: Проверка: попытка добавить пересекающееся назначение завершается ошибкой

```
automasterskaya=# INSERT INTO employee_assignment (
    employee_id,
    category_id,
    appointment_date,
    dismissal_date
)
VALUES (
    (SELECT employee_id FROM employee_assignment ORDER BY assignment_id LIMIT 1),
    (SELECT category_id FROM employee_category ORDER BY category_id LIMIT 1),
    CURRENT_DATE,
    NULL
);
ERROR:  У сотрудника уже есть назначение на должность в указанный период
CONTEXT:  PL/pgSQL function fn_check_employee_assignment_overlap() line 14 at RAISE
.. ■
```

5. Финальная проверка созданных объектов

```
\df pr_*
\df fn_*
```

Рисунок: Список созданных процедур и функций

```
automasterskaya=# \df pr_*
\df fn_*
```

List of functions				
Schema	Name	Result data type	Argument data types	Type
autoservice	pr_increase_ford_parts_price			proc
autoservice	pr_repaired_cars_by_mechanics_last_quarter			proc
autoservice	pr_upgrade_master_grade			proc

(3 rows)

List of functions				
Schema	Name	Result data type	Argument data types	Type
autoservice	fn_audit_contract_line	trigger		func
autoservice	fn_check_contract_administrator	trigger		func
autoservice	fn_check_contract_dates	trigger		func
autoservice	fn_check_contract_line_mechanic	trigger		func
autoservice	fn_check_employee_assignment_overlap	trigger		func
autoservice	fn_fill_labor_price	trigger		func
autoservice	fn_fill_used_part_price	trigger		func

(7 rows)


```

SELECT
    event_object_table AS table_name,
    trigger_name,
    action_timing,
    event_manipulation
FROM information_schema.triggers
WHERE trigger_schema = 'autoservice'
ORDER BY event_object_table, trigger_name;

```

Рисунок: Список созданных триггеров в схеме autoservice

```

automasterskaya=# SELECT
    event_object_table AS table_name,
    trigger_name,
    action_timing,
    event_manipulation
FROM information_schema.triggers
WHERE trigger_schema = 'autoservice'
ORDER BY event_object_table, trigger_name;

```

table_name	trigger_name	action_timing	event_manipulation
contract	trg_check_contract_administrator	BEFORE	INSERT
contract	trg_check_contract_administrator	BEFORE	UPDATE
contract	trg_check_contract_dates	BEFORE	INSERT
contract	trg_check_contract_dates	BEFORE	UPDATE
contract_line	trg_audit_contract_line	AFTER	DELETE
contract_line	trg_audit_contract_line	AFTER	INSERT
contract_line	trg_audit_contract_line	AFTER	UPDATE
contract_line	trg_check_contract_line_mechanic	BEFORE	INSERT
contract_line	trg_check_contract_line_mechanic	BEFORE	UPDATE
contract_line	trg_fill_labor_price	BEFORE	UPDATE
contract_line	trg_fill_labor_price	BEFORE	INSERT
employee_assignment	trg_check_employee_assignment_overlap	BEFORE	INSERT
employee_assignment	trg_check_employee_assignment_overlap	BEFORE	UPDATE
used_part	trg_fill_used_part_price	BEFORE	UPDATE
used_part	trg_fill_used_part_price	BEFORE	INSERT

(15 rows)

6. Вывод

В ходе лабораторной работы были разработаны хранимые процедуры, функции и триггеры для базы данных «Автомастерская». Процедуры автоматизируют изменение цен деталей, повышение разряда мастеров и формирование статистики по ремонту автомобилей за истекший квартал. Триггеры обеспечивают контроль корректности данных, автоматическое заполнение расчетных значений и аудит изменений строк договора. Работоспособность созданных объектов проверена в SQL Shell psql с использованием SQL-запросов и подтверждающих скриншотов.